

Toutes les Fonctions Backend de l'API PrestaShop

8.2.6

Ce document liste TOUTES les fonctions PHP qui sont appelées quand tu fais une requête API. Pour chaque fonction : ce qu'elle fait, dans quel fichier elle se trouve, quand elle est appelée.

Table des matières

1. Cycle de vie d'une requête (résumé visuel)
2. Fonctions du Dispatcher
3. Fonctions de WebserviceRequest (le moteur)
4. Fonctions de WebserviceKey (authentification)
5. Fonctions de WebserviceOutputBuilder (construction réponse)
6. Fonctions de WebserviceOutputXML et JSON (rendu)
7. Fonctions des ObjectModel (entités métier)
8. Fonctions spécifiques Images
9. Fonctions spécifiques Search
10. Fonctions spécifiques Attachments
11. Mapping complet : Méthode HTTP → Fonctions appelées

1. Cycle de vie d'une requête (résumé visuel)

Quand ta NewApp envoie `GET /api/products/1?ws_key=ABC123`, voici TOUTES les fonctions appelées dans l'ordre :

Requête HTTP arrive

```
|
|
|— dispatcher.php
|   |— require config.inc.php           ← Charge PrestaShop
|   |— Context::getContext()           ← Initialise le contexte
|   |— WebserviceKey::getClassFromKey() ← Trouve la classe à utiliser
|   |— WebserviceRequest::getInstance() ← Crée l'instance unique (Singleton)
|
|— WebserviceRequest::fetch()           ← POINT D'ENTRÉE PRINCIPAL
|   |
|   |— webserviceChecks()               ← Vérifications de sécurité
|   |   |— isActivated()                ← PS_WEBSERVICE activé ?
|   |   |— authenticate()              ← Clé API valide ?
|   |   |   |— WebserviceKey::isKeyActive()
|   |   |   |   |— WebserviceKey::getPermissionForAccount()
|   |   |— groupShopExists()           ← Contexte multi-shop OK ?
|   |   |— shopExists()                ← Boutique existe ?
|   |   |— shopHasRight()              ← Clé autorisée sur cette boutique ?
|   |
|   |— checkResource()                  ← "products" existe comme ressource ?
|   |   |— getResources()               ← Charge la liste des 67 ressources
|   |
|   |— checkHTTPMethod()                ← GET autorisé pour "products" ?
|   |
|   |— getOutputObject()                ← XML ou JSON ?
|   |   |— new WebserviceOutputXML() ou new WebserviceOutputJSON()
|   |
|   |— [Selon la méthode HTTP] :
|   |   |— GET → executeEntityGetAndHead()
|   |   |   |— getFilteredObjectDetails() (si /products/1)
|   |   |   |   |— new Product(1)      ← Charge le produit depuis la
|   |   |   |   |   |
|   |   |   |   |   |— getFilteredObjectList() (si /products)
|   |   |   |   |   |   |— manageFilters() ← Construit WHERE, ORDER,
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |— LIMIT
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |— POST → executeEntityPost()
|   |   |   |   |   |   |   |   |— saveEntityFromXml(201)
|   |   |   |   |   |   |   |   |   |— new SimpleXMLElement() ← Parse le XML envoyé
|   |   |   |   |   |   |   |   |   |— new Product()           ← Crée objet vide
|   |   |   |   |   |   |   |   |   |— Product::addWs()        ← Sauvegarde en BDD
|   |   |   |   |   |   |   |   |   |— [traite les associations]
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |— PUT → executeEntityPut()
|   |   |   |   |   |   |   |   |— saveEntityFromXml(200)
|   |   |   |   |   |   |   |   |   |— new Product($id)       ← Charge l'existant
|   |   |   |   |   |   |   |   |   |— Product::updateWs()    ← Met à jour en BDD
|   |   |   |   |   |   |   |   |   |— [traite les associations]
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |— PATCH → executeEntityPatch()
|   |   |   |   |   |   |   |   |— saveEntityFromXml(200)    ← Pareil que PUT mais skip
```

```

les champs absents
|
| |
| | └─ DELETE → executeEntityDelete()
| |         └─ new Product($id)           ← Charge l'objet
| |         └─ Product::delete()         ← Supprime de la BDD
|
| └─ returnOutput()                       ← Construit la réponse finale
|   └─ WebserviceOutputBuilder::getContent()
|     └─ WebserviceOutputXML::renderField() (pour chaque champ)
|   └─ WebserviceOutputBuilder::buildHeader() ← Headers HTTP
|
└─ Réponse envoyée au client

```

2. Fonctions du Dispatcher

Fichier : `prestashop/webservice/dispatcher.php`

Ce n'est pas une classe, c'est un script PHP procédural. Voici ce qu'il exécute :

Ligne	Code	Ce que ça fait
35	<code>require_once config.inc.php</code>	Charge toute la config PrestaShop (BDD, constantes, autoloader)
38	<code>Context::getContext()->cart = new Cart()</code>	Crée un panier vide dans le contexte (nécessaire pour certains calculs de prix)
39	<code>ContainerBuilder::getContainer('webservice')</code>	Initialise le container Symfony pour l'injection de dépendances
40	<code>Currency::getDefaultCurrency()</code>	Charge la devise par défaut
43-46	<code>preg_match('/Basic\s+(.*)\$/i', ...)</code>	Si HTTP Basic Auth, décode le base64 pour extraire la clé
55	<code>\$_REQUEST['ps_method'] ?? \$_SERVER['REQUEST_METHOD']</code>	Détermine la méthode HTTP (permet de simuler PUT via POST)
57-64	<code>\$_SERVER['PHP_AUTH_USER']</code> ou <code>\$_GET['ws_key']</code>	Récupère la clé API
70-76	<code>fopen('php://input', 'rb')</code>	Lit le XML envoyé dans le body (POST/PUT/PATCH)
84	<code>WebserviceKey::getClassFromKey(\$key)</code>	Vérifie si la clé utilise une classe custom (sinon WebserviceRequest)

Ligne	Code	Ce que ça fait
92	<code>WebserviceRequest::getInstance()</code>	Crée le singleton WebserviceRequest
94	<code>\$request->fetch(...)</code>	Exécute toute la logique API
105-109	<code>header(\$param_value)</code>	Envoie les headers HTTP de réponse
113	<code>echo \$result['content']</code>	Affiche le XML/JSON

3. Fonctions de WebserviceRequest (le moteur)

Fichier : `prestashop/classes/webservice/WebserviceRequest.php`

Classe : `WebserviceRequestCore` (1947 lignes)

3.1 — Fonctions d'initialisation

Fonction	Ligne	Description
<code>getInstance()</code>	237	Retourne l'instance unique (Singleton). Si elle n'existe pas, en crée une.
<code>resetStaticCache()</code>	248	Remet l'instance à null (pour les tests).
<code>getOutputObject(\$type)</code>	253	Crée l'objet de rendu (WebserviceOutputXML ou WebserviceOutputJSON) selon le format demandé. Vérifie les headers <code>Io-Format</code> , <code>Output-Format</code> et le paramètre GET <code>output_format</code> .
<code>getResources()</code>	283	Fonction critique : Retourne le tableau de TOUTES les 67 ressources API avec leur classe PHP, description et restrictions. Appelle aussi le hook <code>addWebserviceResources</code> pour que les modules puissent ajouter des ressources.

3.2 — Fonctions de vérification (sécurité)

Fonction	Ligne	Description
<code>fetch(\$key, \$method, \$url, \$params, \$bad_class_name, \$inputXml)</code>	487	LA fonction principale. Orchestre tout : vérifications, exécution, réponse. Appelée par le dispatcher.

Fonction	Ligne	Description
<code>webserviceChecks()</code>	643	Chaîne de vérifications : <code>isActivated() && authenticate() && groupShopExists() && shopExists() && shopHasRight()</code> . Si une seule échoue, tout s'arrête.
<code>isActivated()</code>	863	Vérifie <code>Configuration::get('PS_WEBSERVICE')</code> . Si le webservice est désactivé dans le Back Office → erreur 503.
<code>authenticate()</code>	820	Vérifie la clé API : existe ? active ? 32 caractères ? Charge les permissions. Si échec → erreur 401 + header WWW-Authenticate.
<code>groupShopExists(\$params)</code>	936	Vérifie le paramètre <code>id_shop_group</code> si fourni. Pas obligatoire en mono-boutique.
<code>shopExists(\$params)</code>	903	Vérifie le paramètre <code>id_shop</code> si fourni. Par défaut utilise la boutique du contexte.
<code>shopHasRight(\$key)</code>	879	Requête SQL vers <code>ps_webservice_account_shop</code> pour vérifier que la clé a accès à cette boutique.
<code>checkResource()</code>	984	Parse l'URL pour identifier la ressource demandée. Vérifie qu'elle existe dans <code>getResources()</code> et que la clé a la permission.
<code>checkHTTPMethod()</code>	964	Vérifie que la méthode HTTP est valide (GET/POST/PUT/PATCH/DELETE/HEAD), qu'elle n'est pas dans <code>forbidden_method</code> de la ressource, et que la clé a la permission pour cette méthode.

3.3 — Fonctions d'exécution CRUD

Fonction	Ligne	Description
<code>executeEntityGetAndHead()</code>	1419	Exécute un GET ou HEAD. Si l'URL contient un ID → <code>getFilteredObjectDetails()</code> . Sinon → <code>getFilteredObjectList()</code> .
<code>getFilteredObjectList()</code>	1323	Récupère une LISTE d'objets depuis la BDD. Appelle <code>manageFilters()</code> pour construire le SQL, puis exécute la requête via <code>retrieveMethod</code> (souvent <code>getWebserviceObjectList</code>).
<code>getFilteredObjectDetails()</code>	1356	Récupère UN SEUL objet par son ID. Crée l'objet avec <code>new ClassName(\$id)</code> . Vérifie les droits multi-shop.

Fonction	Ligne	Description
<code>executeEntityPost()</code>	1444	Exécute un POST (création). Appelle <code>saveEntityFromXml(201)</code> .
<code>executeEntityPut()</code>	1454	Exécute un PUT (remplacement complet). Appelle <code>saveEntityFromXml(200)</code> .
<code>executeEntityPatch()</code>	1464	Exécute un PATCH (modification partielle). Appelle <code>saveEntityFromXml(200)</code> . La différence avec PUT : les champs absents du XML ne sont pas modifiés (ligne 1606 : <code>if (\$this->method == 'PATCH' && !isset(\$attributes->{\$fieldName})) continue;</code>).
<code>executeEntityDelete()</code>	1474	Exécute un DELETE. Charge chaque objet par ID, appelle <code>\$object->delete()</code> (ou la méthode custom si définie dans <code>objectMethods['delete']</code>). Support de la suppression multiple via <code>?id=[1,2,3]</code> .
<code>saveEntityFromXml(\$successReturnCode)</code>	1541	Fonction critique : Gère la création ET la modification. Parse le XML avec <code>SimpleXMLElement</code> . Pour chaque entité : crée/charge l'objet, affecte les champs, applique les setters custom, valide, sauvegarde, traite les associations.

3.4 — Fonctions de filtrage et affichage

Fonction	Ligne	Description
<code>manageFilters()</code>	1126	Construit les clauses SQL (WHERE, JOIN, ORDER BY, LIMIT) à partir des paramètres <code>filter</code> , <code>sort</code> , <code>limit</code> , <code>schema</code> de l'URL. Gère aussi les filtres i18n et les linked tables.
<code>setFieldsToDisplay()</code>	1079	Interprète le paramètre <code>display</code> : "full", "minimum", ou "[champ1,champ2]". Vérifie que les champs demandés existent.
<code>parseDisplayFields(\$str)</code>	1035	Parse la syntaxe complexe de <code>display</code> : supporte les associations imbriquées comme <code>display=[id,name,associations[categories[id]]]</code> .
<code>filterLanguage()</code>	1746	Interprète le paramètre <code>language</code> : un ID unique, une liste `[1

Fonction	Ligne	Description
<code>getSQLRetrieveFilter(\$sqlId, \$filterValue, \$tableAlias)</code>	1741	Délègue à <code>SQLUtils::getSQLRetrieveFilter()</code> pour construire les clauses WHERE.

3.5 — Fonctions de sortie

Fonction	Ligne	Description
<code>returnOutput()</code>	1807	Construit la réponse finale. Ajoute les headers (Access-Time, X-Powered-By, Execution-Time, PSWS-Version). Appelle <code>getResourcesList()</code> si pas d'objets, <code>getContent()</code> si objets à afficher, ou <code>getErrors()</code> si erreurs.
<code>setError(\$status, \$label, \$code)</code>	655	Enregistre une erreur (ajoutée au tableau <code>\$this->errors</code>). Met à jour le statut HTTP via <code>objOutput->setStatus()</code> .
<code>setErrorDidYouMean(...)</code>	676	Comme <code>setError()</code> mais suggère une valeur proche (algorithme de Levenshtein). Ex: tu tapes "prodcts" → "Did you mean: products?"
<code>getClosest(\$input, \$words)</code>	689	Calcule la distance de Levenshtein entre l'input et chaque mot pour trouver le plus proche.
<code>webserviceErrorHandler(...)</code>	720	Remplace le error handler PHP par défaut pour afficher les erreurs PHP en format XML au lieu de HTML.
<code>hasErrors()</code>	810	Retourne true si <code>\$this->errors</code> n'est pas vide.

3.6 — Fonctions de prix (spécifiques aux produits)

Fonction	Ligne	Description
<code>getPriceForProduct(\$field, \$entity_object)</code>	380	Calcule le prix d'un produit pour l'affichage. Appelée comme callback par <code>WebserviceOutputBuilder</code> .
<code>specificPriceForProduct(\$entity_object, \$parameters)</code>	398	Calcule le prix spécifique d'un produit en tenant compte du pays, devise, groupe client, quantité...
<code>specificPriceForCombination(\$entity_object, \$parameters)</code>	458	Pareil mais pour une déclinaison (combinaison).

Fonction	Ligne	Description
<code>specificPriceCalculation(\$parameters)</code>	408	Appelle <code>Product::priceCalculation()</code> avec tous les paramètres (shop, country, currency, group, quantity, tax...).

3.7 — Autres fonctions

Fonction	Ligne	Description
<code>getallheaders()</code>	1908	Récupère tous les headers HTTP (compatibilité Apache/Nginx).
<code>setObjectSpecificManagement(\$obj)</code>	1942	Définit le gestionnaire spécifique (images, search, attachments).
<code>setObjects()</code>	1005	Charge les objets par ID(s) depuis l'URL.

4. Fonctions de WebserviceKey (authentification)

Fichier : `prestashop/classes/webservice/WebserviceKey.php`

Classe : `WebserviceKeyCore` (218 lignes)

Fonction	Ligne	Quand est-elle appelée	Ce qu'elle fait
<code>add(\$autodate, \$nullValues)</code>	50	Quand on crée une clé API dans le Back Office	Vérifie que la clé n'existe pas déjà (<code>keyExists</code>), crée l'entrée en BDD, log l'action dans <code>PrestaShopLogger</code> .
<code>keyExists(\$key)</code>	79	Avant de créer une clé	Requête SQL : <code>SELECT key FROM ps_webservice_account WHERE key = ?</code> . Retourne la clé si elle existe, false sinon.
<code>delete()</code>	87	Quand on supprime une clé	Supprime la clé + toutes ses permissions (<code>deleteAssociations</code>). Log l'action.

Fonction	Ligne	Quand est-elle appelée	Ce qu'elle fait
<code>deleteAssociations()</code>	112	Pendant la suppression d'une clé	<code>DELETE FROM ps_webservice_permission WHERE id_webservice_account = ? .</code>
<code>getPermissionForAccount(\$auth_key)</code>	120	Pendant <code>authenticate()</code>	Requête SQL JOIN entre <code>ps_webservice_permission</code> et <code>ps_webservice_account</code> . Retourne un tableau <code>['products' => ['GET', 'POST'], 'customers' => ['GET'], ...]</code> .
<code>isKeyActive(\$auth_key)</code>	141	Pendant <code>authenticate()</code>	<code>SELECT active FROM ps_webservice_account WHERE key = ? .</code> Retourne 1 ou 0.
<code>getClassFromKey(\$auth_key)</code>	152	Dans le dispatcher	<code>SELECT class_name FROM ps_webservice_account WHERE key = ? .</code> Permet d'utiliser une classe custom au lieu de <code>WebserviceRequest</code> .
<code>getIdFromKey(\$auth_key)</code>	165	Quand on a besoin de l'ID	<code>SELECT id_webservice_account WHERE key = ? .</code>
<code>setPermissionForAccount(\$id_account, \$permissions)</code>	182	Quand on modifie les permissions dans le Back Office	Supprime toutes les permissions existantes, puis INSERT les nouvelles. Les méthodes valides sont : GET, PUT, POST, PATCH, DELETE, HEAD.

5. Fonctions de WebserviceOutputBuilder (construction réponse)

Fichier : `prestashop/classes/webservice/WebserviceOutputBuilder.php`

Classe : `WebserviceOutputBuilderCore`

Fonction	Quand est-elle appelée	Ce qu'elle fait
<code>__construct(\$wsUrl)</code>	Dans <code>fetch()</code>	Initialise avec l'URL de base de l'API.
<code>setObjectRender(\$obj)</code>	Dans <code>fetch()</code>	Définit le renderer (XML ou JSON).
<code>setStatus(\$status)</code>	Quand on définit le code HTTP	Stocke le code HTTP (200, 201, 400, 401, 404, 405, 500, 503).
<code>setHeaderParams(\$key, \$value)</code>	Dans <code>returnOutput()</code>	Ajoute un header HTTP (Content-Type, Access-Time, X-Powered-By...).
<code>buildHeader()</code>	Dans <code>returnOutput()</code>	Construit le tableau final de headers HTTP à envoyer.
<code>setWsResources(\$resources)</code>	Dans <code>fetch()</code>	Stocke la liste des ressources (pour générer les xlink).
<code>getResourcesList(\$permissions)</code>	Quand on fait <code>GET /api/</code> (sans ressource)	Génère le XML/JSON listant toutes les ressources disponibles avec leurs descriptions et URLs.
<code>getContent(\$objects, \$schema, \$fields, \$depth, \$viewType)</code>	Dans <code>returnOutput()</code>	Fonction principale de rendu. Selon le viewType (LIST ou DETAILS), génère le contenu complet avec les champs, associations, et xlink.
<code>setSpecificField(\$obj, \$method, \$field, \$resource)</code>	Dans <code>fetch()</code>	Enregistre un callback pour un champ spécifique (ex: calcul de prix).
<code>setVirtualField(\$obj, \$method, \$resource, \$params)</code>	Dans <code>fetch()</code>	Enregistre des champs virtuels (calculés dynamiquement, pas en BDD).
<code>renderErrors(\$errors)</code>	Dans <code>returnOutput()</code>	Formate les erreurs en XML/JSON avec codes et messages.

Constantes importantes :

```
const VIEW_LIST = 1;    // Affichage liste (GET /api/products)
const VIEW_DETAILS = 2; // Affichage détail (GET /api/products/1)
```

6. Fonctions de WebserviceOutputXML et JSON (rendu)

Fichier XML : `prestashop/classes/webservice/WebserviceOutputXML.php`

Fonction	Ce qu'elle fait
<code>renderField(\$field)</code>	Génère un nœud XML pour un champ. Si i18n → ajoute <code><language id="X"></code> . Si xlink → ajoute l'attribut <code>xlink:href</code> .
<code>renderNodeHeader(\$node_name, \$params)</code>	Génère une balise ouvrante XML : <code><product></code> ou <code><products></code> .
<code>renderNodeFooter(\$node_name)</code>	Génère une balise fermante XML : <code></product></code> .
<code>renderAssociationHeader(\$node)</code>	Génère l'ouverture d'une association : <code><associations><categories></code> .
<code>renderAssociationFooter(\$node)</code>	Ferme l'association : <code></categories></code> <code></associations></code> .
<code>overrideContent(\$content)</code>	Enveloppe le contenu dans <code><?xml version="1.0"?><prestashop xmlns:xlink="..."></code> .
<code>setLanguages(\$languages)</code>	Définit les langues à inclure dans le rendu i18n.
<code>renderErrorsHeader()</code> / <code>renderErrorsBody(\$msg, \$code)</code> / <code>renderErrorsFooter()</code>	Formate les erreurs en XML.

Fichier JSON : `prestashop/classes/webservice/WebserviceOutputJSON.php`

Fonction	Ce qu'elle fait
Mêmes fonctions que XML mais génère du JSON au lieu du XML.	
La principale différence : les xlink sont des propriétés d'objet, pas des attributs XML.	

7. Fonctions des ObjectModel (entités métier)

Chaque ressource API est liée à un ObjectModel. Voici les fonctions communes à TOUTES les entités :

Fonctions héritées de `ObjectModel` (classe parente)

Fonction	Quand est-elle appelée	Ce qu'elle fait
<code>__construct(\$id)</code>	GET, PUT, PATCH, DELETE	Charge l'objet depuis la BDD par son ID. Remplit toutes les propriétés.
<code>add(\$autodate, \$nullValues)</code>	POST (via <code>saveEntityFromXml</code>)	INSERT en BDD. Si <code>\$autodate=true</code> , remplit automatiquement <code>date_add</code> et <code>date_upd</code> .
<code>update(\$nullValues)</code>	PUT, PATCH (via <code>saveEntityFromXml</code>)	UPDATE en BDD. Met à jour <code>date_upd</code> .
<code>delete()</code>	DELETE (via <code>executeEntityDelete</code>)	DELETE FROM en BDD.
<code>validateFields(\$die, \$return)</code>	POST, PUT, PATCH (avant sauvegarde)	Vérifie chaque champ selon les règles définies dans <code>\$definition</code> (type, taille, regex, required...).
<code>validateFieldsLang(\$die, \$return)</code>	POST, PUT, PATCH (avant sauvegarde)	Pareil mais pour les champs i18n (multilingues).
<code>getWebserviceParameters(\$wsParamsAttributeName)</code>	Dans <code>fetch()</code> pour charger la config	Retourne le tableau <code>\$webserviceParameters</code> avec les informations de la BDD : table, clé primaire, champs, associations...
<code>getWebserviceObjectList(\$join, \$filter, \$sort, \$limit)</code>	GET (liste)	Requête SQL SELECT pour récupérer la liste des IDs correspondant aux filtres.

Fonctions SPÉCIFIQUES par entité

Product (`classes/Product.php`)

Fonction	Quand	Ce qu'elle fait
<code>addWs()</code>	POST /api/products	Méthode d'ajout personnalisée. Crée le produit + les associations (catégories, fournisseurs...).
<code>updateWs()</code>	PUT/PATCH /api/products	Méthode de mise à jour personnalisée. Met à jour le produit + synchronise les associations.
<code>getCoverWs()</code>	GET (champ <code>id_default_image</code>)	Retourne l'ID de l'image de couverture du produit.
<code>setCoverWs(\$id_image)</code>	POST/PUT/PATCH (champ <code>id_default_image</code>)	Définit quelle image est la couverture.
<code>getWsPositionInCategory()</code>	GET (champ <code>position_in_category</code>)	Retourne la position du produit dans sa catégorie.
<code>setWsPositionInCategory(\$position)</code>	POST/PUT/PATCH (champ <code>position_in_category</code>)	Modifie la position du produit.
<code>getWsType()</code>	GET (champ <code>type</code>)	Retourne "simple", "pack", ou "virtual".
<code>setWsType(\$type)</code>	POST/PUT/PATCH (champ <code>type</code>)	Définit le type du produit.
<code>getWsManufacturerName()</code>	GET (champ <code>manufacturer_name</code>)	Retourne le nom de la marque (lecture seule).

Customer (`classes/Customer.php`)

Fonction	Quand	Ce qu'elle fait
<code>addWs()</code>	POST /api/customers	Crée le client + génère le <code>secure_key</code> .
<code>updateWs()</code>	PUT/PATCH /api/customers	Met à jour le client.

Fonction	Quand	Ce qu'elle fait
<code>setWsPasswd(\$plain_text_password)</code>	POST/PUT/PATCH (champ <code>passwd</code>)	Hash le mot de passe en clair avec bcrypt avant de le stocker. Tu envoies "MonMotDePasse" → stocké comme "2y10\$xxxxx...".

Order (`classes/order/Order.php`)

Fonction	Quand	Ce qu'elle fait
<code>addws()</code>	POST /api/orders	Crée la commande via l'API (méthode custom).
<code>setWsCurrentState(\$id_order_state)</code>	PATCH (champ <code>current_state</code>)	Change le statut de la commande. Déclenche toute la logique métier : envoi d'email au client, génération de facture si applicable, mise à jour de l'historique.
<code>getWsCurrentState()</code>	GET (champ <code>current_state</code>)	Retourne l'ID du statut actuel.
<code>setWsShippingNumber(\$shipping_number)</code>	PUT/PATCH (champ <code>shipping_number</code>)	Définit le numéro de suivi colis.
<code>getWsShippingNumber()</code>	GET (champ <code>shipping_number</code>)	Retourne le numéro de suivi.
<code>getWsOrderRows()</code>	GET (association <code>order_rows</code>)	Retourne les lignes de la commande (produits commandés). Lecture seule (le setter est <code>false</code>).

OrderHistory (`classes/order/OrderHistory.php`)

Fonction	Quand	Ce qu'elle fait
<code>add()</code>	POST /api/order_histories	Ajoute une entrée d'historique ET change le statut de la commande. Peut déclencher l'envoi d'emails.

Cart (`classes/Cart.php`)

Fonction	Quand	Ce qu'elle fait
<code>add()</code>	POST /api/carts	Crée un panier vide en BDD.

Fonction	Quand	Ce qu'elle fait
<code>update()</code>	PUT/PATCH /api/carts	Met à jour le panier (les associations <code>cart_rows</code> sont traitées séparément).

StockAvailable (`classes/stock/StockAvailable.php`)

Fonction	Quand	Ce qu'elle fait
<code>update()</code>	PUT/PATCH /api/stock_availlables	Modifie la quantité en stock d'un produit/déclinaison.

Category (`classes/Category.php`)

Fonction	Quand	Ce qu'elle fait
<code>add()</code>	POST /api/categories	Crée la catégorie + régénère l'arbre NTree.
<code>delete()</code>	DELETE /api/categories	Supprime la catégorie + régénère l'arbre NTree.

Note sur NTree : PrestaShop utilise un algorithme "Nested Tree" pour stocker l'arborescence des catégories. Quand on ajoute/supprime une catégorie, l'arbre doit être recalculé. L'API fait ça automatiquement via `Category::regenerateEntireNtree()` .

8. Fonctions spécifiques Images

Fichier : `prestashop/classes/webservice/WebserviceSpecificManagementImages.php`

Classe : `WebserviceSpecificManagementImagesCore` (1204 lignes)

Les images ne passent PAS par le système ObjectModel classique. Elles ont leur propre gestionnaire.

Fonction	Quand	Ce qu'elle fait
<code>manage()</code>	Toute requête sur /api/images	Point d'entrée. Détermine le type d'image (product, category, manufacturer...) et l'action à effectuer.
<code>setObjectOutput(\$obj)</code>	Initialisation	Stocke la référence vers WebserviceOutputBuilder.
<code>setWsObject(\$obj)</code>	Initialisation	Stocke la référence vers WebserviceRequest.
<code>getContent()</code>	Pour la réponse	Retourne le contenu (XML descriptif ou binaire de l'image).

Types d'images gérés :

```
general    → header, mail, invoice, store_icon
products   → Images des produits (les plus courantes)
categories → Images des catégories
manufacturers → Logos des marques
suppliers  → Logos des fournisseurs
stores     → Photos des magasins
customizations → Images de personnalisation
```

Formats acceptés : GIF, JPG, JPEG, PNG, WebP

9. Fonctions spécifiques Search

Fichier : `prestashop/classes/webservice/WebserviceSpecificManagementSearch.php`

Classe : `WebserviceSpecificManagementSearchCore` (147 lignes)

Fonction	Quand	Ce qu'elle fait
<code>manage()</code>	GET /api/search? query=...	Exécute la recherche dans le catalogue PrestaShop. Utilise le moteur de recherche interne.
<code>setObjectOutput(\$obj)</code>	Initialisation	Stocke WebserviceOutputBuilder.
<code>setWsObject(\$obj)</code>	Initialisation	Stocke WebserviceRequest.
<code>getContent()</code>	Pour la réponse	Retourne les résultats de recherche en XML/JSON.

Restriction : UNIQUEMENT GET. Les méthodes PUT, POST, PATCH, DELETE sont interdites
(`forbidden_method`).

10. Fonctions spécifiques Attachments

Fichier :

`prestashop/classes/webservice/WebserviceSpecificManagementAttachments.php`

Classe : `WebserviceSpecificManagementAttachmentsCore`

Fonction	Quand	Ce qu'elle fait
<code>manage()</code>	Toute requête sur <code>/api/attachments</code>	Gère l'upload/download de pièces jointes (PDF, docs...).
<code>setObjectOutput(\$obj)</code>	Initialisation	Stocke WebserviceOutputBuilder.
<code>setWsObject(\$obj)</code>	Initialisation	Stocke WebserviceRequest.
<code>getContent()</code>	Pour la réponse	Retourne le contenu.

11. Mapping complet : Méthode HTTP → Fonctions appelées

GET /api/{resource} (Liste)

```
dispatcher.php
→ WebserviceRequest::fetch()
→ webserviceChecks() [isActivated, authenticate, shopExists, shopHasRight]
→ checkResource()
→ checkHTTPMethod()
→ getWebserviceParameters() ← Charge la config de la ressource
→ executeEntityGetAndHead()
→ getFilteredObjectList()
→ manageFilters() ← Construit SQL WHERE/ORDER/LIMIT
→ ObjectModel::getWebserviceObjectList() ← Exécute le SELECT
→ new ClassName($id) pour chaque résultat
→ returnOutput()
→ WebserviceOutputBuilder::getContent(objects, null, fields, depth,
VIEW_LIST)
→ WebserviceOutputXML::renderField() pour chaque champ
→ WebserviceOutputBuilder::buildHeader()
```

GET /api/{resource}/{id} (Détail)

```
dispatcher.php
→ WebserviceRequest::fetch()
→ [mêmes vérifications]
→ executeEntityGetAndHead()
  → getFilteredObjectDetails()
    → new ClassName($id) ← Charge l'objet unique
    → Vérifie droits multi-shop
  → returnOutput()
    → WebserviceOutputBuilder::getContent(objects, null, fields, depth,
VIEW_DETAILS)
```

POST /api/{resource} (Création)

```
dispatcher.php
→ WebserviceRequest::fetch()
→ [mêmes vérifications]
→ executeEntityPost()
  → saveEntityFromXml(201)
    → new SimpleXMLElement($inputXml) ← Parse le XML envoyé
    → new ClassName() ← Crée un objet VIDE
    → Pour chaque champ du XML :
      → Si setter custom → $object->setterMethod($value)
      → Sinon → $object->property = $value
    → $object->validateFields() ← Valide les données
    → $object->validateFieldsLang() ← Valide les champs i18n
    → $object->add() ou $object->addWs() ← INSERT en BDD
    → Pour chaque association → traite les relations
  → returnOutput() avec statut 201
```

PUT /api/{resource}/{id} (Remplacement complet)

```
dispatcher.php
→ WebserviceRequest::fetch()
→ [mêmes vérifications]
→ executeEntityPut()
  → saveEntityFromXml(200)
    → new SimpleXMLElement($inputXml)
    → new ClassName($id) ← Charge l'objet EXISTANT
    → Pour TOUS les champs de la config :
      → Si dans le XML → affecte la valeur
      → Si PAS dans le XML → met à null ⚠ (DANGER du PUT)
    → $object->validateFields()
    → $object->update() ou $object->updateWs() ← UPDATE en BDD
  → returnOutput() avec statut 200
```

PATCH /api/{resource}/{id} (Modification partielle)

```
dispatcher.php
→ WebserviceRequest::fetch()
→ [mêmes vérifications]
→ executeEntityPatch()
→ saveEntityFromXml(200)
→ new SimpleXMLElement($inputXml)
→ new ClassName($id) ← Charge l'objet EXISTANT
→ Pour chaque champ de la config :
→ if (PATCH && champ PAS dans le XML) → SKIP  (sécurité du PATCH)
→ Si dans le XML → affecte la valeur
→ $object->validateFields()
→ $object->update() ou $object->updateWs() ← UPDATE en BDD
→ returnOutput() avec statut 200
```

DELETE /api/{resource}/{id} (Suppression)

```
dispatcher.php
→ WebserviceRequest::fetch()
→ [mêmes vérifications]
→ executeEntityDelete()
→ new ClassName($id) ← Charge l'objet
→ Si $id n'existe pas → erreur 404
→ Si objectMethods['delete'] défini → $object->customDelete()
→ Sinon → $object->delete() ← DELETE FROM en BDD
→ Si Category → Category::regenerateEntireNtree()
→ returnOutput() (pas de contenu, juste headers)
```

GET /api/{resource}?schema=blank (Template vide)

```
dispatcher.php
→ WebserviceRequest::fetch()
→ [mêmes vérifications]
→ manageFilters() détecte schema=blank
→ $this->schemaToDisplay = 'blank'
→ returnOutput()
→ WebserviceOutputBuilder::getContent(objects, 'blank', ...)
→ Génère un XML avec tous les champs VIDES (prêt à remplir pour un POST)
```

GET /api/ (Liste des ressources)

```
dispatcher.php
→ WebserviceRequest::fetch()
→ [mêmes vérifications]
→ checkResource() détecte URL vide
→ returnOutput()
→ WebserviceOutputBuilder::getResourcesList($permissions)
→ Génère la liste de toutes les ressources avec descriptions et URLs
```

Résumé : les fonctions les plus importantes à retenir

Rang	Fonction	Pourquoi elle est critique
1	<code>WebserviceRequest::fetch()</code>	Orchestre TOUT le traitement d'une requête
2	<code>WebserviceRequest::saveEntityFromXml()</code>	Gère toute la logique de création/modification
3	<code>WebserviceRequest::getResources()</code>	Définit les 67 ressources disponibles
4	<code>WebserviceRequest::authenticate()</code>	Sécurité : vérifie la clé API
5	<code>ObjectModel::getWebserviceParameters()</code>	Définit quels champs sont exposés
6	<code>WebserviceOutputBuilder::getContent()</code>	Formate la réponse XML/JSON
7	<code>WebserviceKey::getPermissionForAccount()</code>	Charge les droits de la clé
8	<code>Customer::setWsPasswd()</code>	Hash sécurisé du mot de passe
9	<code>Order::setWsCurrentState()</code>	Change le statut + logique métier (emails, factures)
10	<code>Product::addWs()</code> / <code>updateWs()</code>	Création/modification produit avec associations